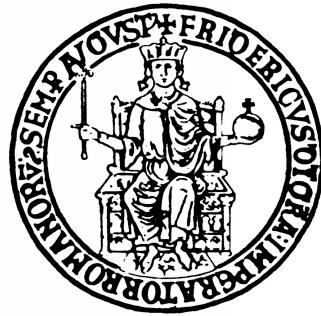




UNIVERSITÀ DI NAPOLI FEDERICO II

Progetto Network Security

Student :

Raffaele Imperato

Alberto Petillo

February 27, 2025

Contents

1	Introduzione	2
1.1	Topologia	2
1.2	Controller	5
2	IDS	8
2.1	Avvio della rete	8
2.2	Generazione del traffico benigno	9
2.3	Generazione del traffico maligno	9
2.4	Algoritmo K-Means	10
2.4.1	Conversione del dataset	10
2.4.2	Scelta del numero di cluster	10
2.4.3	Rilevamento delle anomalie	10
2.4.4	Misura delle metriche di valutazione	10
2.4.5	Risultati ottenuti	10
3	Soluzione proposta	13
3.1	Codice sorgente del controller modificato	13
3.2	Funzionamento	18
4	Conclusioni	19

Chapter 1

Introduzione

Il progetto consiste nella simulazione di una rete virtuale utilizzando la tecnologia SDN (Software-defined networking); questa consiste nella dissociazione del Data Plane dal Control Plane, dove quest'ultimo è costituito da un controller che rappresenta l'elemento centrale della rete.

Il Control Plane diventa centralizzato e non più distribuito, permettendo un controllo unificato e semplificato della rete attraverso un unico punto di controllo. Di conseguenza, i router possono diventare dispositivi più semplici ed economici. All'interno del progetto si andrà a costruire una rete SDN per simulare degli attacchi DoS. Nel contesto di questo progetto, per configurare la rete, è stato configurato un ambiente di virtualizzazione utilizzando VMware. VMware, una delle soluzioni di virtualizzazione più diffuse, è stato utilizzato per eseguire una VM Ubuntu 22.04.

1.1 Topologia

In primo luogo si è deciso di definire la topologia della rete da simulare: a tal proposito è stato utilizzato **Mininet**, un simulatore di rete leggero e flessibile che permette di creare ambienti di rete virtualizzati in modo rapido e realistico. Grazie a Mininet, è stato possibile emulare una rete completa di host e switch all'interno di una singola macchina, semplificando il testing e la validazione delle configurazioni di rete. La topologia è definita mediante uno script Python.

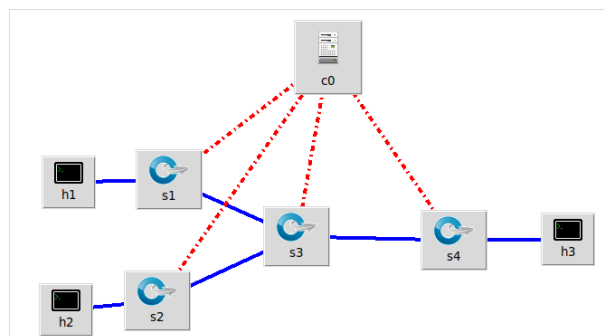


Figure 1.1: Topologia di MiniEdit

Il codice per ottenere la rete richiesta è il seguente:

```
1 from mininet.net import Mininet
```



```
2 from mininet.node import RemoteController, OVSKernelSwitch, Host
3 from mininet.cli import CLI
```

Listing 1.1: Librerie

Qui vengono importati i moduli di Mininet necessari per costruire la topologia della rete:

- **Mininet** è la classe responsabile, come vedremo dopo, della creazione effettiva della rete;
- **RemoteController** permette di interconnettere la rete a un controller remoto per la gestione degli switch;
- **OVSKernelSwitch** è una sottoclasse di Switch che utilizza il kernel Open vSwitch. È un tipo di switch virtuale che si comporta come uno switch Ethernet;
- **Host** è la classe per i nodi host nella rete, i quali rappresentano i computer o dispositivi connessi alla rete;
- La classe **CLI** fornisce un'interfaccia a linea di comando per interagire con la rete Mininet durante l'esecuzione;

Vediamo successivamente come la rete viene definita:

```
1 def myNetwork():
```

Listing 1.2: Definizione della rete

Questo è il metodo effettivo della creazione della rete, che verrà successivamente chiamato nel main.

```
1 net = Mininet( topo=None,
2               build=False,
3               ipBase='10.0.0.0/8')
```

Listing 1.3: Creazione della rete

Notiamo che non viene inizialmente utilizzata una topologia predefinita (*topo=None*): l'andremo a costruire successivamente.

Il parametro *build=False* indica che la rete non viene costruita subito; è necessario chiamare *net.build()* più avanti. Viene poi definito l'indirizzo IP base della rete.

```
1 info('* Adding controller\n')
2 c0 = net.addController(name='c0',
3                       controller=RemoteController,
4                       ip='127.0.0.1',
5                       protocol='tcp',
6                       port=6653)
```

Listing 1.4: Specifica del controller

Qui si sta specificando il nome del controller, il suo IP, il protocollo utilizzato e il porto su cui è collegato.



```
1 info('* Add switches\n')
2 s1 = net.addSwitch('s1', cls=OVSKernelSwitch)
3 s2 = net.addSwitch('s2', cls=OVSKernelSwitch)
4 s3 = net.addSwitch('s3', cls=OVSKernelSwitch)
5 s4 = net.addSwitch('s4', cls=OVSKernelSwitch)
```

Listing 1.5: Aggiunta switch

Successivamente, grazie alla primitiva **addSwitch**, vengono aggiunti quattro switch alla rete, ognuno dei quali è di tipo **Open vSwitch**.

```
1 info('* Add hosts\n')
2 h1 = net.addHost('h1', cls=Host, ip='10.0.0.1', defaultRoute=None)
3 h2 = net.addHost('h2', cls=Host, ip='10.0.0.2', defaultRoute=None)
4 h3 = net.addHost('h3', cls=Host, ip='10.0.0.3', defaultRoute=None)
```

Listing 1.6: Aggiunta degli host

Nel codice precedente, si aggiungono tre host alla rete attraverso il comando `addHost` specificandone l'indirizzo IP.

Andiamo poi a definire i collegamenti effettivi:

```
1 info('* Add hosts\n')
2 info('* Add links\n')
3 net.addLink(h1, s1)
4 net.addLink(h2, s2)
5 net.addLink(h3, s4)
6 net.addLink(s1, s3)
7 net.addLink(s2, s3)
8 net.addLink(s3, s4)
```

Listing 1.7: Creazione dei link

È possibile in realtà andare anche a definire alcuni parametri dei collegamenti, come la bandwidth e il delay; si è deciso però di lasciare quelli di default poiché in visione dell'attacco DoS erano più che ottimali. Questi sono pari rispettivamente a **10 Mbps** per la bandwidth e **0 ms** per il delay; questi link vengono effettuati attraverso la primitiva *addLink*.

```
1 info('* Starting network\n')
2 net.build()
3 info('* Starting controllers\n')
4 for controller in net.controllers:
5     controller.start()
6
7 info('* Starting switches\n')
8 net.get('s1').start([c0])
9 net.get('s2').start([c0])
10 net.get('s3').start([c0])
11 net.get('s4').start([c0])
```

Listing 1.8: Avvio della rete

A questo punto si costruisce la rete effettiva avviando tutti i dispositivi della rete. Lo script itera su tutti i controller avviandoli nonostante ne abbiamo definito uno; questa è una soluzione scalabile nel caso si vogliano utilizzare più controller con una logica diversa. Successivamente, si avviano gli switch e li si collegano al controller.



```
1 info('* Post configure switches and hosts\n')
2 CLI(net)
```

Listing 1.9: Interazione tramite terminale

Infine *CLI(net)*, la quale sta per (Command-Line Interface), che permette di interagire con la rete tramite terminale, per testarne la sua configurazione.

La rete virtuale costruita è quindi composta da tre host e quattro switch collegati tra loro usando un controller che gestisce la rete.

1.2 Controller

Il controller è una componente fondamentale quando si lavora con reti basate su OpenFlow o, più in generale, con reti definite dal software (SDN - Software Defined Networking). Il controller agisce come il "cervello" della rete SDN, dirigendo il flusso del traffico tra i dispositivi di rete.

Mininet permette di creare reti virtuali composte da switch, host e collegamenti tra di essi. In una configurazione SDN, i dispositivi fisici (in questo caso gli switch virtuali OpenFlow) non prendono decisioni autonome su come instradare i pacchetti, ma si affidano alle istruzioni fornite da un controller centrale. Quando si crea uno switch OpenFlow, questo non conosce in anticipo come instradare i pacchetti. Quando arriva un pacchetto, lo switch invia una richiesta al controller, chiedendo cosa fare con quel pacchetto (ad esempio, quale porta usare per inoltrarlo).

Mininet supporta diversi controller SDN che possono essere utilizzati per simulare reti OpenFlow. Nell'esempio dell'elaborato progettuale abbiamo utilizzato Ryu, che è un framework per SDN scritto in Python, progettato per essere facilmente integrabile con applicazioni di rete. Il controller è eseguito in locale, ovvero sulla stessa macchina in cui è in esecuzione Mininet.

Il controller utilizzato nella fase iniziale dell'elaborato è il seguente:

1. Importazione delle librerie:

```
1 from ryu.base import app_manager
2 from ryu.controller import ofp_event
3 from ryu.controller.handler import CONFIG_DISPATCHER,
   MAIN_DISPATCHER
4 from ryu.controller.handler import set_ev_cls
5 from ryu.ofproto import ofproto_v1_3
6 from ryu.lib.packet import packet
7 from ryu.lib.packet import ethernet
8 from ryu.lib.packet import ether_types
```

Listing 1.10: Importazione delle librerie necessarie per il controller Ryu

2. Inizializzazione del controller:

```
1 class SimpleSwitch13(app_manager.RyuApp):
2     OFP_VERSIONS = [ofproto_v1_3.OFP_VERSION]
3
4     def __init__(self, *args, **kwargs):
5         super(SimpleSwitch13, self).__init__(*args, **kwargs)
```



```
6 self.mac_to_port = {}
```

Listing 1.11: Definizione della classe del controller e inizializzazione

3. Gestione delle funzionalità dello switch:

```
1 @set_ev_cls(ofp_event.EventOFPSwitchFeatures, CONFIG_DISPATCHER)
2 def switch_features_handler(self, ev):
3     datapath = ev.msg.datapath
4     ofproto = datapath.ofproto
5     parser = datapath.ofproto_parser
6
7     match = parser.OFPMatch()
8     actions = [parser.OFPActionOutput(ofproto.OFPP_CONTROLLER,
9         ofproto.OFPCML_NO_BUFFER)]
9     self.add_flow(datapath, 0, match, actions)
```

Listing 1.12: Gestione dell'evento di connessione dello switch e aggiunta della regola di flusso

4. Aggiunta di una regola di flusso:

```
1 def add_flow(self, datapath, priority, match, actions, buffer_id=
    None):
2     ofproto = datapath.ofproto
3     parser = datapath.ofproto_parser
4
5     inst = [parser.OFPInstructionActions(ofproto.
        OFPIT_APPLY_ACTIONS, actions)]
6     if buffer_id:
7         mod = parser.OFPFlowMod(datapath=datapath,
8             buffer_id=buffer_id, priority=priority, match=match,
9             instructions=inst)
10    else:
11        mod = parser.OFPFlowMod(datapath=datapath,
12            priority=priority, match=match, instructions=inst)
13    datapath.send_msg(mod)
```

Listing 1.13: Funzione per aggiungere una regola di flusso allo switch

5. Gestione dei pacchetti "Packet-In":

```
1 @set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
2 def _packet_in_handler(self, ev):
3     # If you hit this you might want to increase
4     # the "miss_send_length" of your switch
5     if ev.msg.msg_len < ev.msg.total_len:
6         self.logger.debug("packet truncated: only %s of %s bytes",
7             ev.msg.msg_len,
8             ev.msg.total_len)
9     msg = ev.msg
10    datapath = msg.datapath
11    ofproto = datapath.ofproto
12    parser = datapath.ofproto_parser
13    in_port = msg.match['in_port']
14
15    pkt = packet.Packet(msg.data)
```



```
15     eth = pkt.get_protocols(ethernet.ethernet)[0]
16
17     if eth.ethertype == ether_types.ETH_TYPE_LLDP:
18         # ignore lldp packet
19         return
20     dst = eth.dst
21     src = eth.src
22
23     dpid = datapath.id
24     self.mac_to_port.setdefault(dpid, {})
25
26     self.logger.info("packet in %s %s %s %s", dpid, src, dst,
27                     in_port)
28
29     # learn a mac address to avoid FLOOD next time.
30     self.mac_to_port[dpid][src] = in_port
31
32     if dst in self.mac_to_port[dpid]:
33         out_port = self.mac_to_port[dpid][dst]
34     else:
35         out_port = ofproto.OFPP_FLOOD
36
37     actions = [parser.OFPACTIONOutput(out_port)]
38
39     # install a flow to avoid packet_in next time
40     if out_port != ofproto.OFPP_FLOOD:
41         match = parser.OFPMATCH(in_port=in_port, eth_dst=dst,
42                                 eth_src=src)
43         # verify if we have a valid buffer_id, if yes avoid to send
44         # both
45         # flow_mod & packet_out
46         if msg.buffer_id != ofproto.OFP_NO_BUFFER:
47             self.add_flow(datapath, 1, match, actions, msg.
48                           buffer_id)
49             return
50         else:
51             self.add_flow(datapath, 1, match, actions)
52     data = None
53     if msg.buffer_id == ofproto.OFP_NO_BUFFER:
54         data = msg.data
55
56     out = parser.OFPPACKETOut(datapath=datapath,
57                               buffer_id=msg.buffer_id, in_port=in_port,
58                               actions=actions, data=data)
59     datapath.send_msg(out)
```

Listing 1.14: Gestione dei pacchetti "Packet-In" ricevuti dallo switch

Chapter 2

IDS

Per rilevamento di anomalie si intende il metodo di determinazione di eventi rari che si discostano significativamente dai modelli o dai comportamenti tipici. Questa tecnica è infatti utilizzata per individuare comportamenti illeciti su Internet, come gli attacchi informatici. Le anomalie possono essere trovate tramite algoritmi di Machine Learning, per consentire una risposta tempestiva ed efficace. Il clustering, in particolare, rappresenta un insieme di tecniche di apprendimento non supervisionate, utilizzate per organizzare le istanze di un dataset in gruppi, raccogliendole per similitudine. Nel contesto della rilevazione delle anomalie, l'idea di base è considerare le istanze che cadono all'interno dei gruppi come normali, mentre quelle che cadono al di fuori di questi gruppi sono considerate anomalie. Per tale motivo andremo ad addestrare l'IDS sul traffico regolare della rete. Nel seguito vedremo come è stata effettuata la creazione del dataset per l'addestramento di un algoritmo K-Means.

2.1 Avvio della rete

Avviato l'ambiente virtuale Linux e il terminale, si è proceduto con l'esecuzione dei seguenti comandi:

Listing 2.1: Comando per deallocare le risorse di una topologia mininet precedente

```
sudo mn -c
```

Prima di ogni esecuzione ci si è assicurato di resettare l'ambiente Mininet creato precedentemente. Successivamente:

Listing 2.2: Comando per avviare il controller

```
python3.9 -m ryu.cmd.manager controller.py
```

Il comando viene utilizzato per avviare un'applicazione Ryu SDN controller.

Per avviare poi la topologia, si utilizza il seguente comando:

Listing 2.3: Comando per avviare la topologia

```
sudo python3 topology.py
```

All'interno dell'ambiente è possibile effettuare diverse operazioni; per esempio, con il comando *mininet> pingall* è possibile verificare tutte connessioni della rete, utile successivamente. In seguito, per ogni host, si è avviato il rispettivo terminale, utilizzando il comando:



Listing 2.4: Comando per avviare il terminale degli host

```
xterm h1 h2 h3
```

Per la raccolta del traffico di rete è stato utilizzato Wireshark.

2.2 Generazione del traffico benigno

Il traffico benigno è stato utilizzato per addestrare l'algoritmo K-Means e quindi identificare i cluster al cui interno andranno a ricadere i pacchetti relativi allo scambio di messaggi regolari nella rete. Di seguito sono riportati i comandi eseguiti su ciascun terminale:

Listing 2.5: Terminale di h1

```
iperf 10.0.0.2 10.0.0.3
```

Listing 2.6: Terminale di h2

```
iperf 10.0.0.1 10.0.0.3
```

Listing 2.7: Terminale di h3

```
iperf 10.0.0.1 10.0.0.2
```

2.3 Generazione del traffico maligno

Successivamente h1 e h2 sono stati utilizzati per fare un attacco DDos su h3. Gli attacchi utilizzati sono:

- **TCP SYN Flood:**

Listing 2.8: Terminale di h1 e h2

```
hping3 -S -p 80 -c 200000 10.0.0.3
```

- **UDP Flood:** si basa sull'invio di pacchetti UDP verso alcuni numeri di porto del target, dato che UDP non controlla ne flusso né congestione.

Listing 2.9: Terminale di h3

```
ITGRecv
```

Listing 2.10: Terminale di h1 e h2

```
ITGSend -T UDP -a 10.0.0.3 -t 120000 -C 200000 -c 200000 -l  
udp_att
```

2.4 Algoritmo K-Means

Per lo sviluppo dell'algoritmo è stata utilizzata la libreria Scikit-Learn. Nella prima fase sono stati ottenuti il modello K-Means per il dataset benigno e un valore di soglia basato sulla distanza massima tra i punti dati e il relativo centroide. Nella fase successiva è stato analizzato il dataset maligno sul modello trovato precedentemente, i punti che superano la soglia vengono considerati come anomalie. Infine sono state misurate le metriche di valutazione: Accuracy, Precision, Recall ed F1-Score.

2.4.1 Conversione del dataset

Il dataset in esame comprende pacchetti provenienti dal traffico di rete catturati con Wireshark. La tabella è composta da righe e colonne, dove rispettivamente si trovano le istanze del dataset e le loro caratteristiche. Le features utilizzate sono: tempo di arrivo, IP Sorgente, IP Destinazione, protocollo e lunghezza; erano presenti anche il numero del pacchetto e le informazioni associate, che sono state scartate. Sono stati poi rimossi dal dataset i pacchetti con protocolli MAC e IPV6, dato che gli attacchi erano tutti incapsulati in pacchetti IPV4. Infine è stato necessario standardizzare le caratteristiche dei dati per renderle omogenee.

2.4.2 Scelta del numero di cluster

Per la scelta ottimale del numero di cluster sono stati valutati con il metodo della silhouette, che ha suggerito la divisione del dataset in 6 cluster.

2.4.3 Rilevamento delle anomalie

Il cluster 3 è quello a cui appartengono le anomalie, di conseguenza quando viene valutato un punto appartenente a questo cluster vengono attivate le contromisure necessarie a contrastare la minaccia.

2.4.4 Misura delle metriche di valutazione

L'algoritmo K-Means è stato eseguito su un dataset etichettato, per cui i dati etichettati come anomalie dall'algoritmo sono stati poi confrontati con le etichette in modo da poter valutare le prestazioni dell'algoritmo tramite le metriche di valutazione Accuracy, Precision, Recall e F1-Score.

2.4.5 Risultati ottenuti

I risultati dell'algoritmo rappresentano un elemento fondamentale per valutare l'efficacia e l'efficienza della soluzione proposta. Essi forniscono una panoramica delle prestazioni dell'algoritmo in termini di Accuracy, Precision, Recall e F1-Score, consentendo di trarre conclusioni significative per l'utilizzo dell'anomaly detection nel contesto di attacchi informatici. Le metriche di valutazione sono definite come segue:

- **Accuracy:** è la parte della previsione che il nostro modello ha fatto in modo corretto. Può essere calcolata come segue:

$$\frac{TP + TN}{TP + TN + FP + FN}$$

- **Precision:** è l'accuratezza con cui il sistema di machine learning prevede le classi positive. Il valore di Precision elevato significa pochi falsi positivi. Questo valore può essere calcolato come segue:

$$\frac{TP}{TP + FP}$$

- **Recall:** rappresenta la frazione di anomalie effettivamente rilevate dal modello rispetto a tutte le anomalie presenti nei dati. Misura l'efficacia del modello nel trovare tutte le anomalie. Un valore di Recall elevato significa pochi falsi negativi. Questo valore si misura come:

$$\frac{TP}{TP + FN}$$

- **F1-Score:** rappresenta la media armonica di Precision e Recall. In particolare l'F1-Score fornisce una valutazione equilibrata delle prestazioni del modello, soprattutto quando c'è uno squilibrio tra dati normali e anomali. Un punteggio F1 elevato indica un buon compromesso tra Precision e Recall. Si può calcolare nel seguente modo:

$$\frac{2 \cdot Precision \cdot Recall}{Precision + Recall}$$

Dove:

- TP = Veri positivi, ovvero il numero di campioni positivi correttamente classificati;
- TN = Veri negativi, ovvero il numero di campioni negativi correttamente classificati;
- FP = Falsi positivi, ovvero il numero di campioni negativi erroneamente classificati;
- FN = Falsi negativi, ovvero il numero di campioni positivi erroneamente classificati.

Metrica di Valutazione	Score
Accuracy	0.99367
Precision	0.99997
Recall	0.98769
F1-Score	0.99379

Table 2.1: Valutazione dell'accuratezza dell'algoritmo di K-Means secondo le metriche di valutazione Accuracy, Precision, Recall, F1-Score.

I risultati degli esperimenti sono riportati in Table 2.1.



I risultati ottenuti indicano che il modello ha un'alta precisione, il che significa che la stragrande maggioranza delle istanze etichettate come eventi anomali sono effettivamente attacchi. La Recall è anche elevata, suggerendo che il modello è in grado di individuare una grande parte degli attacchi reali. L'F1 Score, essendo vicino a 1, suggerisce che il modello ha un buon equilibrio tra precisione e richiamo.

In generale, questi risultati sembrano molto promettenti e indicano che, in questo contesto, il modello K-Means è risultato efficace nel rilevare gli attacchi DDoS eseguiti dall'attaccante.

Chapter 3

Soluzione proposta

E' stato scelto di affidare il compito di individuare le anomalie al controller della rete SDN, in quanto essendo connesso a tutti gli switch esso può ricevere il traffico in ingresso e analizzare i pacchetti. In tempo reale il controller dunque andrà a classificare ciascun pacchetto ricevuto e, se risulta anomalo, incrementerà un contatore per quell'host che raggiunta una soglia andrà a bloccare quel mittente. Ovviamente il confronto è basato sul modello e la soglia ottenuti dall'algoritmo k-means addestrato sul traffico regolare della rete.

3.1 Codice sorgente del controller modificato

1. Importazione delle librerie:

```
1 from datetime import datetime
2 import random
3 import joblib
4 import numpy as np
5 from ryu.base import app_manager
6 from ryu.controller import ofp_event
7 from ryu.controller.handler import CONFIG_DISPATCHER,
    MAIN_DISPATCHER
8 from ryu.controller.handler import set_ev_cls
9 from ryu.ofproto import ofproto_v1_3
10 from ryu.lib.packet import packet, ethernet, ether_types, ipv4, tcp
    , udp
11 from ryu.lib import hub
```

Listing 3.1: Importazione delle librerie

2. Dizionari per protocolli e indirizzi IP: Questi dizionari sono necessari per effettuare le predizioni con il modello addestrato, utilizzati per convertire le features dei pacchetti raccolti dal controller nello stesso formato utilizzato durante l'addestramento.

```
1 protocolsDict = {
2     "UDP" : 1,
3     "IPv4" : 2,
4     "TCP" : 3,
5     "ARP" : 4,
6     "ICMPv6" : 5,
```



```
7     "MDNS" : 6
8 }
9
10 ipDict = {
11     "10.0.0.1" : 1,
12     "10.0.0.2" : 2,
13     "10.0.0.3" : 3,
14 }
```

Listing 3.2: Dizionari per protocolli e IP

3. Classe principale ‘SimpleSwitchIDS’:

```
1 class SimpleSwitchIDS(app_manager.RyuApp):
2     OFP_VERSIONS = [ofproto_v1_3.OFP_VERSION]
3
4     def __init__(self, *args, **kwargs):
5         super(SimpleSwitchIDS, self).__init__(*args, **kwargs)
6         self.mac_to_port = {}           # Per il MAC learning
7         self.datapaths = {}             # Per salvare i datapath
8                                         # degli switch connessi
9         self.counters = {}              # Per contare i pacchetti
10                                         # sospetti per IP
11
12         # Inizializzo il modello
13         self.kmeans_model = joblib.load('Dataset/modello_e_soglia.
14                                         pkl')
15         self.threshold = 10             # Soglia per rilevamento
16                                         # attacco
17
18         # Avvia un thread per resettare i contatori dei pacchetti
19         # ogni 10 secondi
20         self.reset_thread = hub.spawn(self._reset_counters)
```

Listing 3.3: Classe principale ‘SimpleSwitchIDS’

4. Gestione degli eventi e aggiunta delle flow entries di base:

```
1 @set_ev_cls(ofp_event.EventOFPSwitchFeatures, CONFIG_DISPATCHER)
2 def switch_features_handler(self, ev):
3     datapath = ev.msg.datapath
4     self.datapaths[datapath.id] = datapath
5     parser = datapath.ofproto_parser
6     ofproto = datapath.ofproto
7
8     # Installazione della flow entry di table-miss
9     match = parser.OFPMatch()
10    actions = [parser.OFPActionOutput(ofproto.OFPP_CONTROLLER,
11                                     ofproto.OFPCML_NO_BUFFER)]
12    self.add_flow(datapath, priority=0, match=match, actions=
13                  actions)
14
15    # Tutti i pacchetti SYN devono andare al controller
16    match = parser.OFPMatch(eth_type=0x0800, ip_proto=6, tcp_flags
17                             =0x02) # TCP SYN
18    actions = [parser.OFPActionOutput(ofproto.OFPP_CONTROLLER)]
```



```
16 self.add_flow(datapath, priority=100, match=match, actions=
    actions)
```

Listing 3.4: Gestione degli eventi e aggiunta delle flow entries

5. **Funzioni di IDS per il controllo dei pacchetti:** La funzione **Syn Check** elabora il pacchetto in ingresso per capire se è un pacchetto SYN, costruendo un dizionario per effettuare una predizione con il modello. Sostanzialmente i pacchetti con un flag SYN senza flag ACK sono considerati sospetti. La funzione **UDP Check** elabora il pacchetto in ingresso per capire se è un pacchetto UDP allo stesso modo inizializzando i valori che non sono specificati da UDP. In entrambi i casi viene aggiornato il valore del contatore dei pacchetti sospetti provenienti dalla sorgente IP indicata nel pacchetto. In caso il numero di pacchetti superi una certa soglia stabilita, viene richiamata la funzione di blocco del traffico proveniente dall'IP sospetto.

```
1 def _syn_check(self, tcp_pkt, ip_pkt, pkt, datapath):
2     features = {
3         "No.": tcp_pkt.seq,
4         "Time": datetime.now().timestamp(),
5         "Source": ip_pkt.src,
6         "Destination": ip_pkt.dst,
7         "Protocol": ip_pkt.proto,
8         "Lenght": len(pkt),
9         "ACK": 1 if (tcp_pkt.bits & 0x10) else 0, # Flag ACK
10        "SYN": 1 if (tcp_pkt.bits & 0x02) else 0, # Flag SYN
11    }
12    if features["Source"] in ipDict: features["Source"] = ipDict[
13        features["Source"]]
14    if features["Destination"] in ipDict: features["Destination"] =
15        ipDict[features["Destination"]]
16    if features["Protocol"] in protocolsDict: features["Protocol"] =
17        protocolsDict[features["Protocol"]]
18
19    features = np.array(list(features.values())).reshape(1, -1)
20    prediction = self.kmeans_model["kmeans_model"].predict(features
21    )
22
23    if prediction[0] == 3:
24        self.counters.setdefault(ip_pkt.src, 0)
25        self.counters[ip_pkt.src] += 1
26        if self.counters[ip_pkt.src] > self.threshold:
27            self.logger.warning(f"Pacchetto sospetto da {ip_pkt.src
28            }, bloccato.")
29            self._block_ip(datapath, ip_pkt.src)
30            return
31
32    def _udp_check(self, ip_pkt, pkt, datapath):
33        """
34        Estraggo dal pacchetto IP le caratteristiche rilevanti
35        per il modello
36        """
37        features = {
38            "No.": random.randint(1, 1000000),
39            "Time": datetime.now().timestamp(),
40            "Source": ip_pkt.src,
41            "Destination": ip_pkt.dst,
42            "Protocol": ip_pkt.proto,
```



```
36         "Lenght": len(pkt),
37         "ACK": 0,
38         "SYN": 0
39     }
40     if features["Source"] in ipDict: features["Source"] =
ipDict[features["Source"]]
41     if features["Destination"] in ipDict: features["
Destination"] = ipDict[features["Destination"]]
42     if features["Protocol"] in protocolsDict: features["
Protocol"] = protocolsDict[features["Protocol"]]
43     #self.logger.info(features)
44     features = np.array(list(features.values())).reshape(1, -1)
45
46     # Predizione con il modello KMeans
47     prediction = self.kmeans_model["kmeans_model"].predict(
features)
48     self.logger.info(prediction)
49
50     if prediction[0] == 3:
51         self.counters.setdefault(ip_pkt.src, 0)
52         self.counters[ip_pkt.src] += 1
53         if self.counters[ip_pkt.src] > self.threshold:
54             self.logger.warning(f"Pacchetto sospetto da {ip_pkt
.src}, bloccato.")
55             self._block_ip(datapath, ip_pkt.src)
56             return
```

Listing 3.5: Funzioni di IDS per il controllo dei pacchetti

6. **Funzione di blocco dell'IP:** Vengono aggiunte due flow entry ad altissima priorità che bloccano tutto il traffico dall'IP sospetto, compresi i pacchetti SYN, con un timeout di 600 secondi.

```
1 def _block_ip(self, datapath, ip_src):
2     parser = datapath.ofproto_parser
3     match = parser.OFPMatch(eth_type=0x0800, ipv4_src=ip_src)
4     actions = [] # Nessuna azione: i pacchetti verranno scartati
5     self.add_flow(datapath, priority=1000, match=match, actions=
actions, idle_timeout=600)
6
7     match2 = parser.OFPMatch(eth_type=0x0800, ip_proto=6, tcp_flags
=0x02, ipv4_src=ip_src)
8     self.add_flow(datapath, priority=1000, match=match2, actions=
actions, idle_timeout=600)
9
10    self.logger.info("Bloccato traffico da IP %s su DPID %s",
ip_src, datapath.id)
```

Listing 3.6: Funzione di blocco dell'IP



7. Reset dei contatori:

```
1 def _reset_counters(self):
2     """
3     Thread che ogni 10 secondi resetta i contatori dei pacchetti
4     malevoli.
5     """
6     while True:
7         hub.sleep(10)
8         self.counters.clear()
9         self.logger.info("Reset contatori.")
10
11     # Richiede le statistiche per ogni datapath
12     for datapath in self.datapaths.values():
13         self.request_port_stats(datapath)
```

Listing 3.7: Reset dei contatori

Chapter 4

Conclusioni

In conclusione, l'implementazione di un sistema di rilevamento delle anomalie (IDS) in una rete SDN, integrato nel controller e basato su un modello di clustering K-Means, ha dimostrato di essere una soluzione efficace per il monitoraggio e la protezione della rete. L'analisi del traffico proveniente dagli switch, utilizzando tecniche di Machine Learning, ha consentito di identificare e classificare con precisione comportamenti anomali, garantendo elevati livelli di performance del sistema. I risultati ottenuti, con metriche come l'accuracy e la precisione superiori al 95%, confermano l'affidabilità e l'efficacia di questa architettura nel rilevamento precoce di minacce. Inoltre, l'integrazione dell'IDS nel controller SDN ha reso possibile una gestione centralizzata e dinamica della sicurezza della rete, rispondendo in tempo reale alle problematiche di sicurezza. La centralizzazione del controller SDN rappresenta un vantaggio in termini di gestione e monitoraggio centralizzato, ma allo stesso tempo introduce una serie di vulnerabilità e rischi associati alla sua posizione strategica. Poiché tutte le decisioni di gestione della rete, inclusi i meccanismi di sicurezza come l'IDS, dipendono da un singolo punto di controllo rendendo il controller a tutti gli effetti un single point of failure. Un attacco mirato al controller potrebbe compromettere l'intero sistema di rilevamento delle anomalie. Un malfunzionamento o un attacco riuscito al controller potrebbe disabilitare o manipolare il funzionamento del sistema IDS, riducendo la capacità della rete di rilevare attacchi o anomalie. Inoltre, poiché il controller gestisce l'intera rete, un sovraccarico di richieste o l'elaborazione di grandi quantità di traffico potrebbero compromettere le sue prestazioni, causando ritardi nella rilevazione delle anomalie o, nei casi più gravi, una perdita di funzionalità nel sistema di sicurezza. Pertanto, la centralità del controller impone un attento bilanciamento tra la potenza computazionale necessaria per l'analisi del traffico e la robustezza del sistema contro potenziali attacchi esterni o guasti interni.